

Trabajo Práctico Integrador

Gestión de Datos de Países en Python

Alumnos

Nayla Raiolo, Alma Acosta

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación I

Docente Titular

Cinthia Rigoni (Comisión 1)

Ariel Enferrel (Comision 11)

Docente Tutor

Martin Garcia (Comision 1)

Luciano Chiroli (Comisión 11)

11 de Noviembre de 2025

Tabla de contenido

Consignas	3
Desarrollo	
Consigna 1	5
Consigna 2	6
Conclusión	8
Referencias	9

Introducción

El presente trabajo práctico integrador tiene como objetivo desarrollar una aplicación en **Python 3** que permita gestionar información sobre distintos países, aplicando estructuras de datos como listas y diccionarios, junto con funciones, condicionales, filtros, ordenamientos y estadísticas básicas. El sistema fue diseñado para leer datos desde un archivo **CSV**, procesarlos y brindar al usuario un menú interactivo que facilita la búsqueda, actualización y análisis de la información almacenada.

A través de este proyecto buscamos afianzar los conocimientos adquiridos en la materia **Programación I**, fomentando la modularización del código, la validación de datos y el uso de técnicas de procesamiento de información. Además, el desarrollo promueve la comprensión práctica de conceptos fundamentales de la programación estructurada, preparando al grupo para abordar problemas reales mediante soluciones organizadas y eficientes.

Por último, este trabajo representa una oportunidad para integrar contenidos teóricos con la práctica, fortaleciendo la lógica de programación, la autonomía en el desarrollo de software y el trabajo colaborativo en entornos virtuales. De esta manera, el proyecto contribuye al aprendizaje significativo y al desarrollo de competencias esenciales en el campo de la tecnología.

Objetivos

El presente trabajo busca alcanzar los siguientes propósitos específicos:

- Aplicar estructuras de datos como listas y diccionarios para almacenar y manipular información.
- Implementar funciones modulares que permitan una organización clara del código.
- Utilizar condicionales y estructuras repetitivas para controlar el flujo del programa.
- Desarrollar un menú interactivo que facilite la navegación del usuario.
- Incorporar métodos de validación de datos para asegurar la integridad de la información.
- Generar estadísticas básicas (promedios, máximos, mínimos y conteo) a partir del dataset.
- Fomentar el trabajo autónomo y la aplicación práctica de los conceptos aprendidos en la materia Programación I.

Desarrollo

Marco Teórico

El desarrollo del presente trabajo práctico integrador se sustenta en diversos conceptos fundamentales de la programación estructurada, abordados en la materia Programación I.

A continuación, se detallan los principales temas aplicados en la implementación del sistema.

- **Listas**

Las listas son estructuras de datos dinámicas que permiten almacenar múltiples valores bajo un mismo nombre. En Python, se utilizan para agrupar elementos relacionados, facilitando su acceso, modificación y recorrido. En este proyecto, las listas se emplean para contener el conjunto de países leídos desde el archivo CSV, posibilitando realizar operaciones como búsquedas, filtros y ordenamientos de forma sencilla.

Ejemplo:

```
paises = ["Argentina", "Brasil", "Japón"]
```

```
print(paises[0]) # Muestra: Argentina
```

- **Diccionarios**

Los diccionarios son estructuras que almacenan información en pares *clave–valor*. Cada país del sistema se representa mediante un diccionario, donde las claves son los campos (nombre, población, superficie, continente). Este formato permite acceder rápidamente a la información específica de cada país y facilita las actualizaciones o comparaciones dentro de las funciones del programa.

Ejemplo:

```
pais = {"nombre": "Argentina", "poblacion": 45376763, "continente": "América"}
```

```
print(pais["nombre"]) # Muestra: Argentina
```

- **Funciones**

Las funciones son bloques de código reutilizables que agrupan instrucciones con un propósito específico. Su uso favorece la modularización, mejora la legibilidad del programa y evita la repetición de código. En este trabajo, cada funcionalidad —como cargar el CSV, agregar un país, ordenar, filtrar o calcular estadísticas— fue desarrollada en una función independiente, cumpliendo con el principio de una responsabilidad por función.

Ejemplo:

```
def mostrar_menu():
```

```
    print("1. Buscar país")
```

```
    print("2. Agregar país")
```

- **Estructuras condicionales**

Las estructuras condicionales permiten ejecutar diferentes acciones según se cumplan o no determinadas condiciones. Se aplican, por ejemplo, para validar entradas del usuario, verificar si un país ya existe o determinar los criterios de filtrado. Su utilización asegura que el programa responda de forma lógica y controlada ante distintas situaciones.

Ejemplo:

```
if pais["poblacion"] > 50000000:
```

```
    print("País con alta población")
```

```
else:
```

```
    print("País con población baja")
```

- **Estructuras repetitivas**

Las estructuras repetitivas, como los bucles `for` y `while`, posibilitan ejecutar un conjunto de instrucciones varias veces. En el proyecto, se utilizan para recorrer las listas de países, mostrar resultados en pantalla y mantener activo el menú principal hasta que el usuario decida salir.

Ejemplo:

`for pais in paises:`

`print(pais["nombre"])`

- **Ordenamientos y filtros**

Los ordenamientos permiten organizar los países según un criterio específico (por nombre, población o superficie), mientras que los filtros seleccionan únicamente aquellos registros que cumplen ciertas condiciones (por continente o por rango). Estas operaciones son esenciales para el análisis de datos y se implementan mediante expresiones condicionales y funciones integradas de Python como `sorted()` y las *list comprehensions*.

Ejemplo ordenamientos:

```
países = [  
  
    {"nombre": "Argentina", "poblacion": 45376763},  
  
    {"nombre": "Brasil", "poblacion": 213993437},  
  
    {"nombre": "Japón", "poblacion": 125800000}  
  
]
```

Ordenar por población de menor a mayor

```
países_ordenados = sorted(países, key=lambda p: p["poblacion"])
```

```
for país in países_ordenados:
```

```
    print(país["nombre"], país["poblacion"])
```

Ejemplo filtros:

```
países = [
```

```
    {"nombre": "Argentina", "continente": "América"},
```

```
    {"nombre": "Japón", "continente": "Asia"},
```

```
    {"nombre": "Alemania", "continente": "Europa"}]
```

```
# Filtrar países del continente América
```

```
america = [p for p in países if p["continente"] == "América"]
```

```
for país in america:
```

```
    print(país["nombre"])
```

- **Estadísticas básicas**

Las estadísticas básicas brindan información general sobre el conjunto de datos. En este sistema se calculan el país con mayor y menor población, el promedio de población y superficie, y la cantidad de países por continente. Estas métricas permiten interpretar de forma cuantitativa la información contenida en el archivo.

Ejemplo:

```
poblaciones = [45376763, 213993437, 83149300]
```

```
promedio = sum(poblaciones) / len(poblaciones)
```

```
print("Promedio de población:", promedio)
```

- **Archivos CSV**

Los archivos CSV (Comma Separated Values) son un formato estándar para almacenar datos en forma de texto separado por comas. Python facilita su manipulación mediante el módulo csv, que permite leer y escribir registros de manera estructurada. En este trabajo, el programa carga los países desde un archivo `paises_base.csv`, procesa su contenido y permite guardar los cambios realizados por el usuario.

Ejemplo:

```
import csv
```

```
with open("paises_base.csv", "r", encoding="utf-8") as archivo:
```

```
    lector = csv.DictReader(archivo)
```

```
    for fila in lector:
```

```
        print(fila)
```

Síntesis

En conjunto, estos conceptos conforman la base teórica que permitió el desarrollo del sistema de gestión de países. La correcta aplicación de listas, diccionarios, funciones y estructuras de control permitió crear una herramienta modular, clara y eficiente, que refleja los principios esenciales de la programación estructurada.

Caso Práctico

El caso práctico consiste en el desarrollo de un sistema de gestión de datos geográficos implementado en Python.

El programa realiza la lectura inicial del archivo `países_base.csv`, que contiene información estandarizada sobre distintos países.

Su contenido se almacena en una lista de diccionarios, estructura que permite acceder y modificar los datos de manera eficiente.

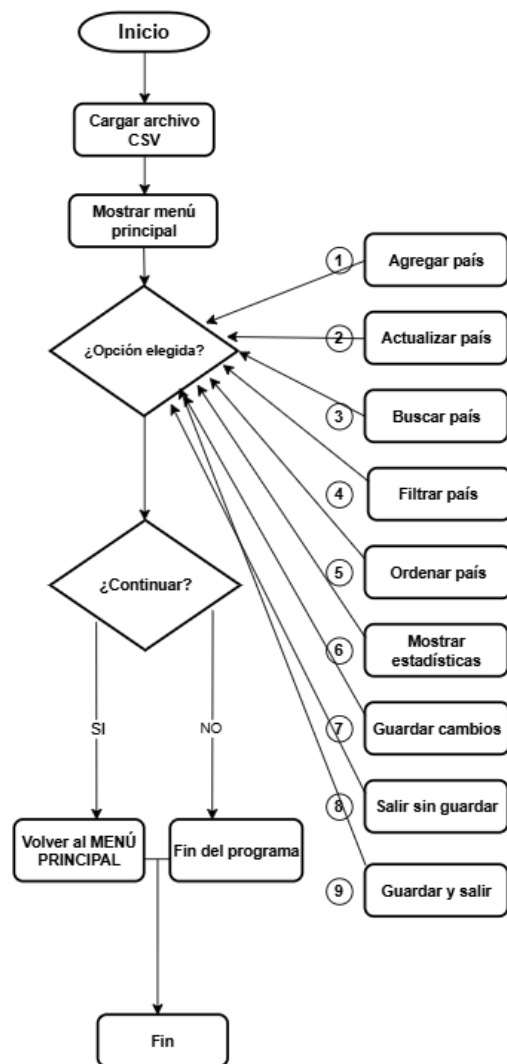
El sistema presenta un menú interactivo que posibilita realizar operaciones de búsqueda, filtrado, ordenamiento, incorporación de nuevos registros y obtención de estadísticas descriptivas.

La aplicación se ejecuta bajo un enfoque modular, organizando cada funcionalidad en funciones independientes para favorecer la flexibilidad, escalabilidad y mantenimiento del código.

Diagrama de flujo

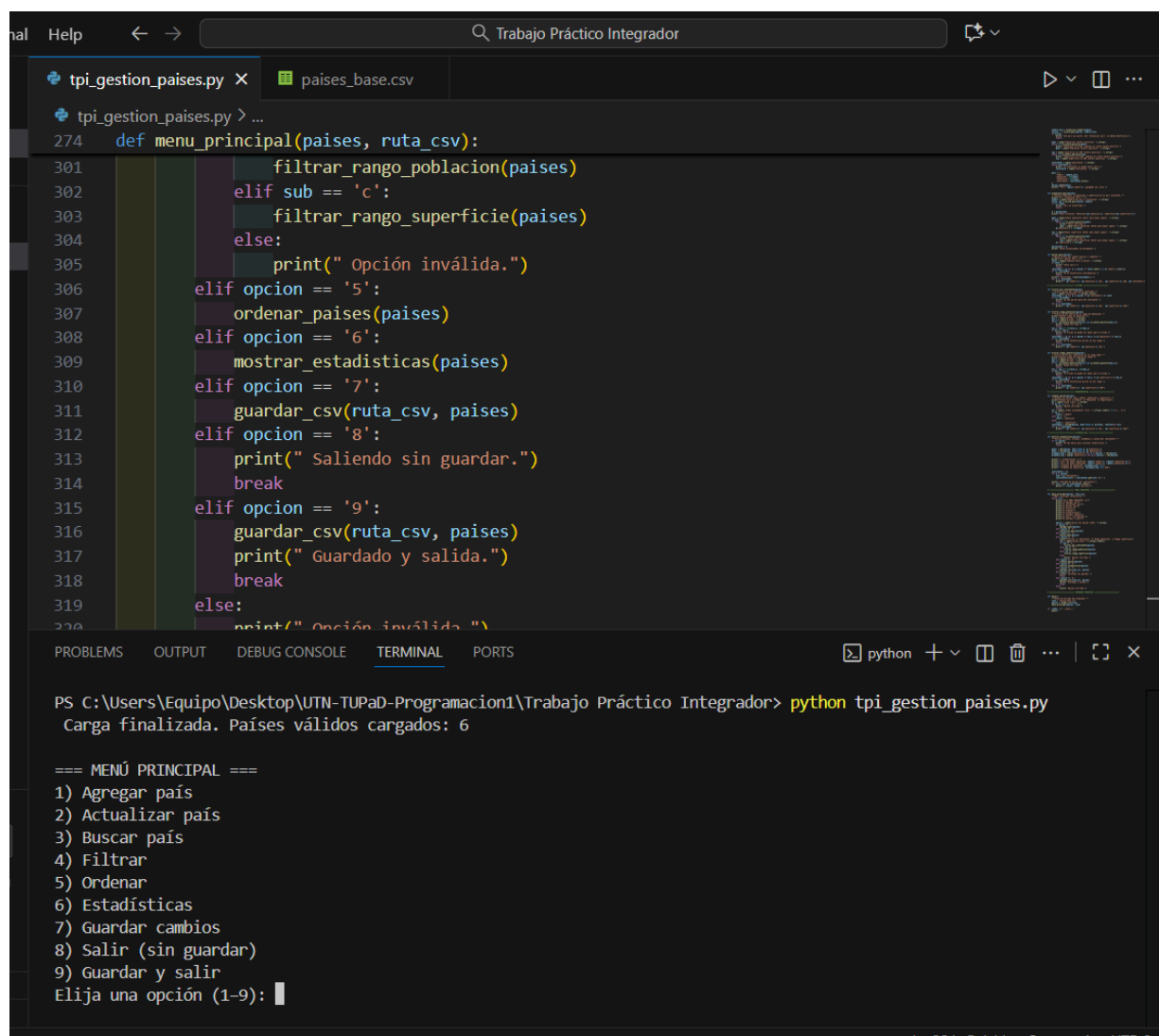
El siguiente diagrama representa el flujo principal de operaciones del sistema.

El programa comienza con la carga de datos desde el archivo CSV y, a través de un menú interactivo, permite al usuario realizar distintas acciones como agregar, buscar, filtrar, ordenar y analizar países. El sistema se ejecuta en un bucle que se repite hasta que el usuario elige salir, garantizando un uso dinámico y continuo.



Capturas de ejecución

Menú principal del sistema.



The screenshot displays a code editor with two files: `tpi_gestion_paises.py` and `paises_base.csv`. The `tpi_gestion_paises.py` file contains a function `menu_principal` that handles various operations on a list of countries. The terminal window shows the execution of the program, which loads 6 valid countries and displays the main menu.

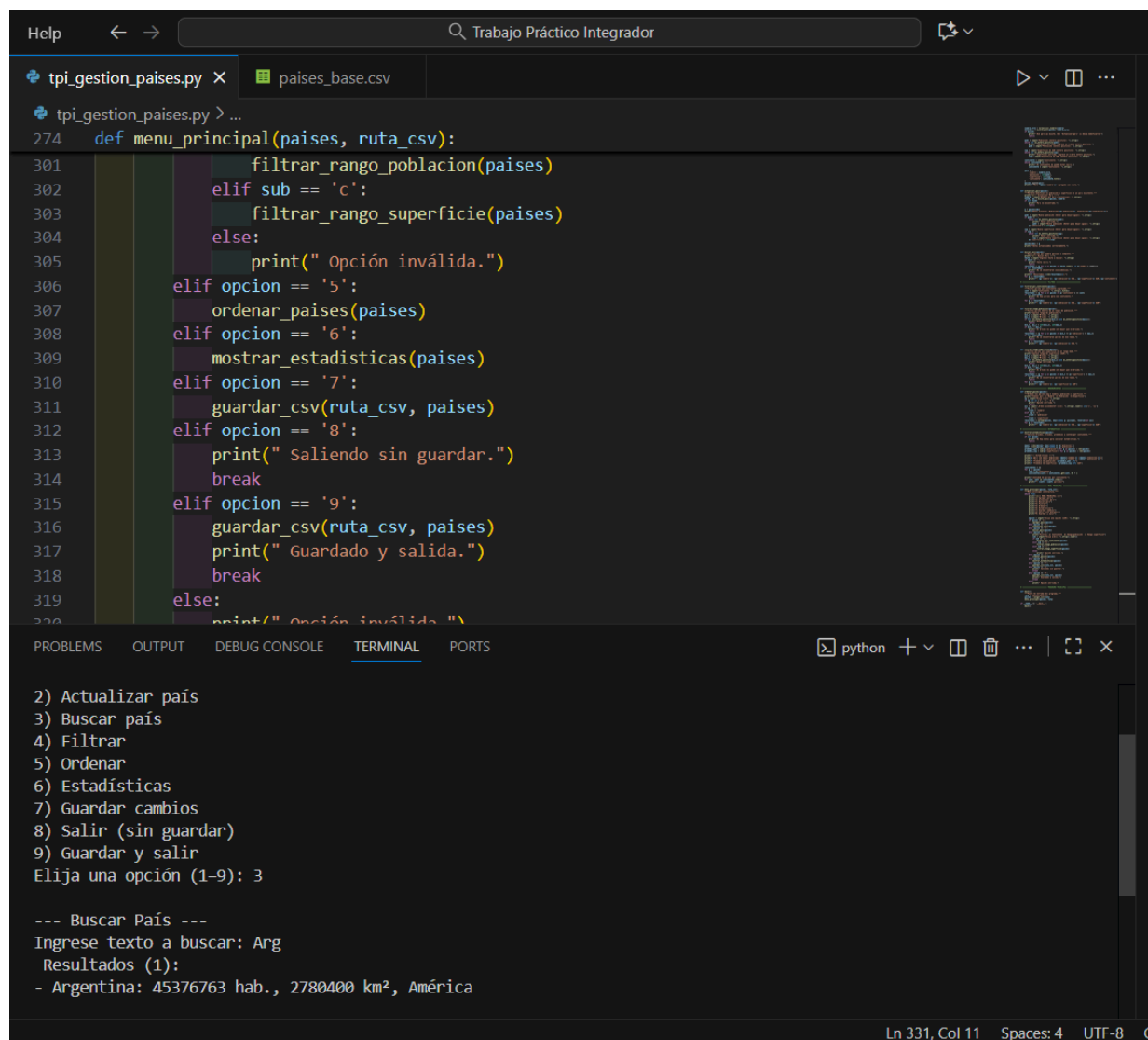
```
def menu_principal(paises, ruta_csv):
    274     filtrar_rango_poblacion(paises)
    275     elif sub == 'c':
    276         filtrar_rango_superficie(paises)
    277     else:
    278         print(" Opción inválida.")
    279     elif opcion == '5':
    280         ordenar_paises(paises)
    281     elif opcion == '6':
    282         mostrar_estadisticas(paises)
    283     elif opcion == '7':
    284         guardar_csv(ruta_csv, paises)
    285     elif opcion == '8':
    286         print(" Saliendo sin guardar.")
    287         break
    288     elif opcion == '9':
    289         guardar_csv(ruta_csv, paises)
    290         print(" Guardado y salida.")
    291         break
    292     else:
    293         print(" Opción inválida.")
```

Terminal Output:

```
PS C:\Users\Equipo\Desktop\UTN-TUPaD-Programacion1\Trabajo Práctico Integrador> python tpi_gestion_paises.py
Carga finalizada. Países válidos cargados: 6

=== MENÚ PRINCIPAL ===
1) Agregar país
2) Actualizar país
3) Buscar país
4) Filtrar
5) Ordenar
6) Estadísticas
7) Guardar cambios
8) Salir (sin guardar)
9) Guardar y salir
Elija una opción (1-9):
```


Ejemplo de búsqueda de un país por nombre.



The image shows a code editor with a Python script named `tpi_gestion_paises.py` and a CSV file named `paises_base.csv`. The script defines a `menu_principal` function that handles various operations on a list of countries. The terminal window shows the program's execution, displaying a menu of options and the results of a search for 'Arg'.

```
274 def menu_principal(paises, ruta_csv):
301     filtrar_rango_poblacion(paises)
302     elif sub == 'c':
303         filtrar_rango_superficie(paises)
304     else:
305         print(" Opción inválida.")
306     elif opcion == '5':
307         ordenar_paises(paises)
308     elif opcion == '6':
309         mostrar_estadisticas(paises)
310     elif opcion == '7':
311         guardar_csv(ruta_csv, paises)
312     elif opcion == '8':
313         print(" Saliendo sin guardar.")
314         break
315     elif opcion == '9':
316         guardar_csv(ruta_csv, paises)
317         print(" Guardado y salida.")
318         break
319     else:
320         print(" Opción inválida ")
```

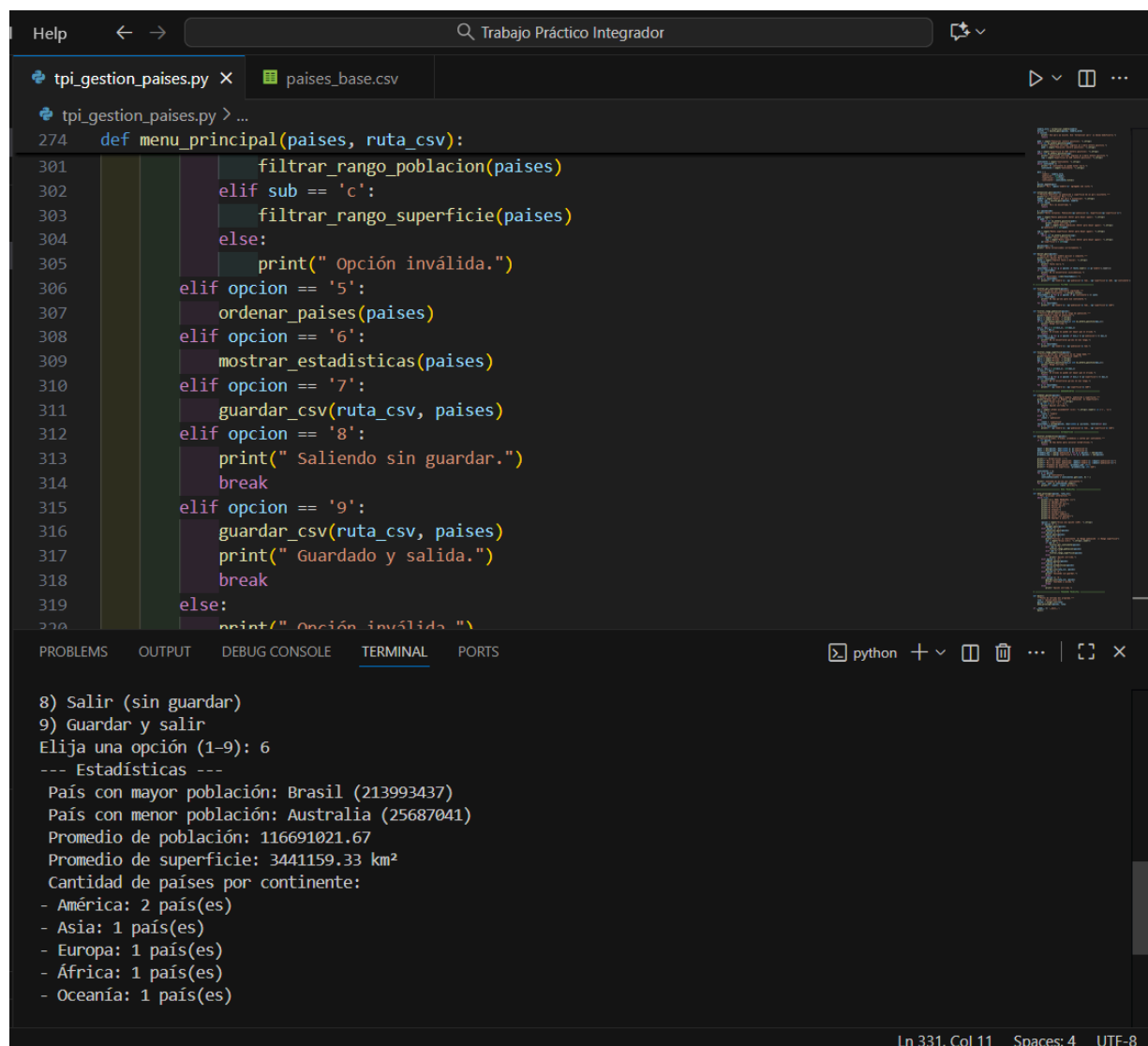
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
2) Actualizar país
3) Buscar país
4) Filtrar
5) Ordenar
6) Estadísticas
7) Guardar cambios
8) Salir (sin guardar)
9) Guardar y salir
Elija una opción (1-9): 3

--- Buscar País ---
Ingrese texto a buscar: Arg
Resultados (1):
- Argentina: 45376763 hab., 2780400 km², América
```

Ln 331, Col 11 Spaces: 4 UTF-8 C

Visualización de estadísticas de población y superficie.



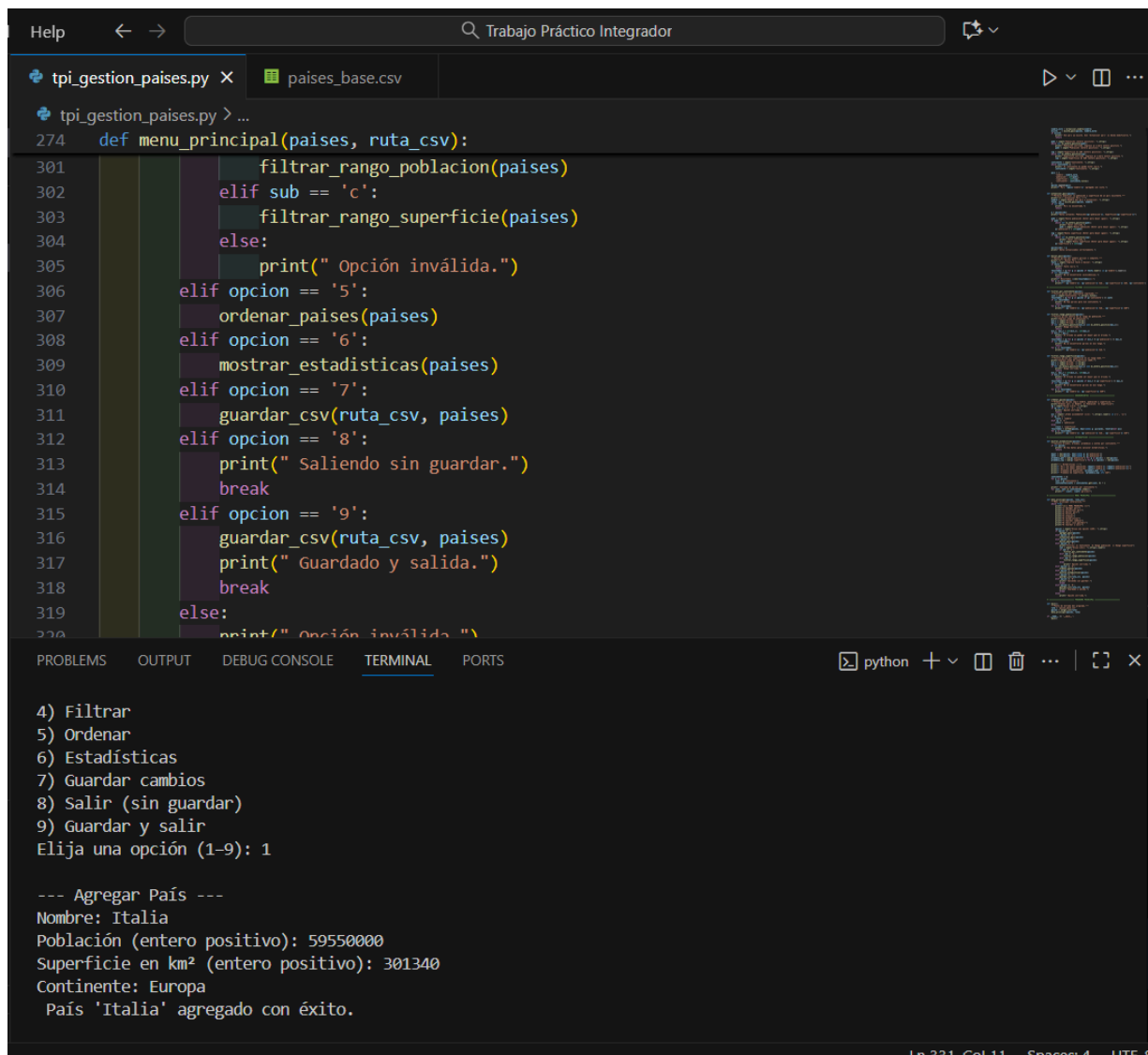
The image shows a code editor with two tabs: `tpi_gestion_paises.py` and `paises_base.csv`. The `tpi_gestion_paises.py` tab is active, displaying a Python function `menu_principal` that handles a menu for managing countries. The function includes options for filtering by population or area, sorting, showing statistics, saving to CSV, and exiting. The terminal window at the bottom shows the execution of the program, where option 6 was selected to view statistics. The output displays various statistics including the most and least populated countries, average population and area, and the number of countries per continent.

```
def menu_principal(paises, ruta_csv):
    while True:
        sub = input("Filtrar por: ")
        if sub == 'p':
            filtrar_rango_poblacion(paises)
        elif sub == 'c':
            filtrar_rango_superficie(paises)
        else:
            print(" Opción inválida.")
        opcion = input("Opción: ")
        if opcion == '5':
            ordenar_paises(paises)
        elif opcion == '6':
            mostrar_estadisticas(paises)
        elif opcion == '7':
            guardar_csv(ruta_csv, paises)
        elif opcion == '8':
            print(" Saliendo sin guardar.")
            break
        elif opcion == '9':
            guardar_csv(ruta_csv, paises)
            print(" Guardado y salida.")
            break
        else:
            print(" Opción inválida.")
```

```
8) Salir (sin guardar)
9) Guardar y salir
Elija una opción (1-9): 6
--- Estadísticas ---
País con mayor población: Brasil (213993437)
País con menor población: Australia (25687041)
Promedio de población: 116691021.67
Promedio de superficie: 3441159.33 km²
Cantidad de países por continente:
- América: 2 país(es)
- Asia: 1 país(es)
- Europa: 1 país(es)
- África: 1 país(es)
- Oceanía: 1 país(es)
```

Ln 331, Col 11 Spaces: 4 UTF-8

Carga de un nuevo país y mensaje de confirmación



The image shows a code editor window with two tabs: `tpi_gestion_paises.py` and `paises_base.csv`. The `tpi_gestion_paises.py` tab is active, displaying a Python function `menu_principal` that handles a menu for managing countries. The function includes options for filtering by population or area, sorting, showing statistics, saving changes, and exiting. The terminal output at the bottom shows the execution of the program, where the user selects option 9 to add a new country. The program prompts for the country name, population, area, and continent, and then confirms the successful addition of 'Italia'.

```
274 def menu_principal(paises, ruta_csv):
301     filtrar_rango_poblacion(paises)
302     elif sub == 'c':
303         filtrar_rango_superficie(paises)
304     else:
305         print(" Opción inválida.")
306     elif opcion == '5':
307         ordenar_paises(paises)
308     elif opcion == '6':
309         mostrar_estadisticas(paises)
310     elif opcion == '7':
311         guardar_csv(ruta_csv, paises)
312     elif opcion == '8':
313         print(" Saliendo sin guardar.")
314         break
315     elif opcion == '9':
316         guardar_csv(ruta_csv, paises)
317         print(" Guardado y salida.")
318         break
319     else:
320         print(" Opción inválida.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
4) Filtrar
5) Ordenar
6) Estadísticas
7) Guardar cambios
8) Salir (sin guardar)
9) Guardar y salir
Elija una opción (1-9): 1

--- Agregar País ---
Nombre: Italia
Población (entero positivo): 59550000
Superficie en km² (entero positivo): 301340
Continente: Europa
País 'Italia' agregado con éxito.
```

Ln 331, Col 11 Spaces: 4 Lf5:8

Metodología utilizada

Aproximación inicial al problema

El desarrollo del presente proyecto comenzó con una revisión exhaustiva del enunciado previsto por la materia con el objetivo de interpretar correctamente los requerimientos funcionales y no funcionales del sistema. Durante la etapa inicial, analizamos las necesidades centrales de la aplicación: gestionar información estructurada de países, permitir operaciones de búsqueda, filtrado y ordenamiento, calcular estadísticas descriptivas y mantener persistencia mediante el uso de archivos en formato CSV.

A partir del análisis, confeccionamos un inventario de funcionalidades y las clasificamos según su prioridad y complejidad técnica. Este proceso permitió determinar una estrategia de desarrollo iterativo, enfocada en construir el sistema de manera progresiva, asegurando la estabilidad de cada módulo antes de avanzar a la siguiente etapa.

Estrategia de desarrollo incremental

Adoptamos una estrategia de desarrollo basada en iteraciones sucesivas, lo que permitió detectar errores de forma temprana y garantizar la funcionalidad parcial en cada avance:

- **Iteración 1:** implementación de la función de lectura del archivo `países_base.csv`. Se generó la estructura inicial de datos utilizando listas de diccionarios y se verificó la correcta carga y visualización.
- **Iteración 2:** diseño del menú principal y primeras funcionalidades básicas, como búsqueda por nombre y listado general. Durante esta etapa se incorporaron validaciones para prevenir errores del usuario.
- **Iteración 3:** desarrollo de funcionalidades avanzadas: filtrado por atributos, ordenamiento por diferentes criterios, estadísticas descriptivas y agregado de nuevos registros.
- **Iteración 4:** refinamiento final: estandarización del estilo de código, mejora de mensajes del sistema, organización coherente de funciones y limpieza de fragmentos redundantes.

Este enfoque permitió mantener un sistema continuamente funcional, facilitando el ajuste gradual de características.

Diseño de estructuras de datos

Se evaluaron tres alternativas para la presentación de los países:

Alternativa	Motivo de descarte o selección
Listas paralelas	Generan dificultad para mantener sincronía entre atributos
Tuplas	Carecían de claridad semántica y mutabilidad
Diccionario	Proveen claridad, mutabilidad y fácil acceso por clave.

Finalmente, se utiliza una lista de diccionarios, donde cada diccionario representa un país con las claves:

{“nombre”: ..., “población”: ..., “superficie”: ..., “continente”: ...}

Esta estructura facilita las operaciones de búsqueda, filtrado, ordenamiento y análisis.

Subsistema de persistencia

La persistencia de datos se implementó mediante el manejo de archivos CSV. Se utilizó el módulo csv de python y se especificó la codificación utf-8 para evitar conflictos con caracteres acentuados.

Se desarrollaron dos funciones principales:

- **Cargar_paises()** = Lee el archivo y carga los datos en memoria.
- **Guardar_paises()** = actualiza el archivo cuando el usuario decide guardar.

Sistema de validación de datos

Se aplicaron múltiples estrategias para asegurar la integridad del sistema:

- Validación de tipo (con try-except)
- Validación de rangos lógicos para datos numéricos.
- Verificación de unicidad para evitar duplicados.
- Validación de dominio para campos restringidos(como continentes válidos)

Los mensajes de error fueron redactados en lenguaje claro para mejorar la experiencia del usuario.

Protocolo de pruebas y aseguramiento de calidad

El sistema fue sometido a pruebas en cuatro categorías:

1. **Pruebas funcionales:** confirmación del correcto funcionamiento de búsquedas, filtros, estadísticas y ordenamientos.
2. **Caso límite:** manejo de listas vacías, extremos numéricos y búsquedas sin resultados.
3. **Pruebas con datos inválidos:** detección de errores de ingreso y prevención de fallos en ejecución.
4. **Pruebas de persistencia:** verificación del guardado y recuperación de la información.

Cada error identificado fue corregido mediante un ciclo de prueba -> corrección -> nueva prueba, asegurando estabilidad en la versión final.

Conclusiones

El desarrollo de este trabajo práctico integrador nos permitió poner en práctica los conocimientos adquiridos en la materia **Programación I**, especialmente en el manejo de estructuras de datos, funciones, condicionales y archivos. A través del proyecto comprendimos cómo organizar un programa de manera modular, clara y eficiente, aplicando la lógica de la programación estructurada a un caso concreto de gestión de información.

Este trabajo también nos ayudó a reforzar la importancia de la validación de datos y del control de errores para garantizar la correcta ejecución del sistema. Además, incorporamos técnicas de filtrado, ordenamiento y análisis estadístico, comprendiendo cómo pueden utilizarse para obtener información útil a partir de grandes volúmenes de datos.

En términos generales, consideramos que el trabajo resultó una experiencia enriquecedora que fortalece nuestras habilidades de razonamiento lógico, resolución de problemas y trabajo colaborativo. Asimismo, consolidó la base teórica y práctica necesaria para continuar avanzando en la carrera y enfrentar futuros proyectos de programación con mayor seguridad y autonomía.

Referencias

<https://docs.python.org/3/tutorial/>

<https://docs.python.org/3/tutorial/datastructures.html>

<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

<https://docs.python.org/3/tutorial/controlflow.html>

<https://docs.python.org/3/library/csv.html>

<https://docs.python.org/3/library/functions.html>

<https://docs.python.org/3/howto/sorting.html>

Diccionarios:

<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

Comprensiones de listas:

<https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions>

Lectura y escritura de archivos

<https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>

<https://docs.python.org/3/library/csv.html>